

Netra

Integrated CCTV Monitoring, Video Analytics and Real-Time Alert Platform — heterogeneous camera integration, AI event correlation, watchlist alerting, GIS movement tracing and a credible path to ~80,000 cameras.

Prepared for	okDriver Smart Dashcams Private Limited
Candidate	Aditya Kaushik · Full Stack Developer (Intern)
Source code	github.com/AdityaKaushik11/netra
Aligned with	Gujarat Police Innovation Hackathon 2026 problem statement
Document date	27 September 2026 · version 1.0

1	Executive summary	9	Real-time architecture
2	Requirement coverage	10	Data model
3	System architecture	11	API design
4	Components & responsibilities	12	Security architecture
5	Deployment & network topology	13	Reliability & operations
6	Camera integration & video path	14	Scalability to 80,000 cameras
7	AI event ingestion pipeline	15	Technology, cost & roadmap
8	Watchlist correlation & alerting		

All figures in this document describe the working prototype in the repository; capacity and cost figures in section 14 are planning estimates with stated assumptions.

1 Executive summary

Netra is a working prototype of a centralised video-intelligence platform. Cameras of different types and vendors are onboarded into one registry, monitored continuously, viewed through one console, enriched with AI inference results, correlated against a watchlist and turned into real-time alerts that an operator can act on — including the vehicle's movement history on a map.

5 logically different camera source types behind one adapter interface	< 1 s detection → alert on every operator screen (WebSocket push)	38 automated tests incl. security, dedup, correlation, retention	₹0 entirely free, open-source software; 0 known dependency vulnerabilities
--	--	--	--

Design principles

Principle	How it shows up in the architecture
Adapters, not special cases	RTSP, ONVIF, HLS, vendor REST APIs and dashcams implement one interface (prepare · probe · playback · teardown). Adding a vendor SDK is one class; nothing else changes.
Move metadata, not pixels	Analytics run at the edge and send ~0.6 KB events; video is pulled through a gateway only when someone watches it. This is what makes 80,000 cameras affordable.
Stateless, horizontally scalable API	Real-time fan-out goes through Redis Pub/Sub, so any API replica can serve any WebSocket; background work (health probing) is leader-elected so it does not multiply with replicas.
Exactly-once semantics where it matters	Idempotency keys, a temporal de-duplication window and alert cool-down stop retries, repeated ANPR reads and alert storms from creating duplicate records or operator noise.
Secure by default	Every request authenticated; camera credentials encrypted and never sent to browsers; video access through short-lived, camera-scoped tokens; SSRF guard; strict CSP; services bound to localhost unless deliberately exposed; everything audited.
Operable for months unattended	Health checks, restart policies, memory limits, bounded logs, data retention, nightly backups and automatic certificate renewal are part of the design, not an afterthought.

End-to-end flow (brief §11) — all steps implemented and verified

#	Step	Where it happens
1	Admin adds two or more camera sources	Console form, bulk API (POST /cameras/bulk , API key) or ONVIF discovery
2	Metadata appears in registry and map	Registry page + Leaflet map; pushed live via camera.created
3	Camera status is monitored	Health monitor: gateway readiness & bitrate, manifest probes, vendor status API, edge heartbeats
4	A live or recorded feed is viewable	LL-HLS / WebRTC via gateway, direct HLS, vendor snapshots; MP4 replay of any recorded window
5–6	Analytics produces an event; backend validates & stores it	REST, batch or Redis Stream → ingestion pipeline → MySQL

7– 8	Event checked against watchlist; match raises a real-time alert	In-memory exact + fuzzy index → alert → WebSocket toast, sound, map pulse
9	Operator searches the entity and sees movement history	Trace page: chronological stops, distances, speeds, route map, cloned-plate anomaly
10	Auditable history of events and alerts	Append-only <code>audit_logs</code> + persisted events/alerts, searchable and exportable (CSV)

2 Requirement coverage

Every section of the brief mapped to the component that implements it.

Brief section	Implementation
§3 Camera registry	Add / edit / disable / search / filter; ID, name, department, lat/long, type, protocol, endpoint reference, status, last heartbeat, zone, storage tier / location / retention; manual, API and ONVIF-discovery onboarding; Online / Degraded / Offline health; per-camera audit history; interactive map.
§4 Live video	RTSP cameras (authenticated) relayed by a MediaMTX gateway; ONVIF Profile S device (SOAP + WS-Security digest); existing HLS publisher; vendor NVR REST API; vehicle dashcam over 4G with GPS. LL-HLS, WebRTC (WHEP) and recorded MP4 playback.
§5 Dashboard	KPI strip (online/degraded/offline, open alerts by severity, detections, ingest rate, watchlist size, duplicates suppressed), live map, alert queue, live camera grid, streaming detections, hourly trend, camera & service health, camera filter, global entity search.
§6 GIS & movement	Status-coloured markers that pulse on open alerts; moving dashcam marker with GPS trail; trace page with numbered chronological route, leg distance / time / implied speed and impossible-travel detection; alert response map showing the operator and the wanted vehicle.
§7 AI analytics	ANPR, vehicle, person, face-recognition, object and dashcam ADAS events (overspeed, harsh braking, collision warning, drowsiness); schema + semantic validation; persistence; correlation; real-time dashboard updates.
§8 Watchlist & alerts	Synthetic watchlist (vehicles and persons); exact and OCR-tolerant fuzzy matching; alerts store timestamp, camera, location, entity, confidence, status; acknowledge / resolve / false-positive, individually or in bulk; alert view shows live feed and a recorded evidence clip.
§9 Real-time	WebSocket over Redis Pub/Sub; Redis Streams consumer group as a message channel; persisted events; idempotency, temporal dedup and alert cool-down against duplicate or uncontrolled events.
§10 Stack	FastAPI / Python, MySQL 8 (RDS-compatible), Redis, WebSocket, WebRTC, React + TypeScript, Leaflet, Docker Compose, OpenAPI / Swagger.
§12 Scalability	Edge / regional / central design, horizontal scaling, bandwidth, indexing & partitioning, queues, GPU sizing, hot / warm / cold storage, monitoring, HA / DR, segmentation, cost estimate (section 14).
§13 Security	JWT + RBAC, server-side logout, account lockout, hashed scoped API keys, encrypted stream credentials, camera-scoped stream tokens, SSRF guard, CSP, HTTPS, rate limiting, audit log, env-based secrets (section 12).
§14 Deliverables	Repository, README, this architecture document, ER diagram, API documentation (Swagger + docs/API.md), representative feeds and watchlist, deployment and scalability notes, demo script.
§18 Optional	ONVIF mock & discovery · RTSP→HLS/WebRTC gateway · ANPR duplicate suppression · temporal correlation · heartbeat service · role-based views · route reconstruction · low-bandwidth snapshot mode · Docker Compose · tests & CI · Prometheus metrics · Redis event architecture · multi-department registry.

3 System architecture

Sources on the left, the platform in the middle, operators on the right. Video, API/data, events and control traffic travel on separate, explicitly authorised paths.

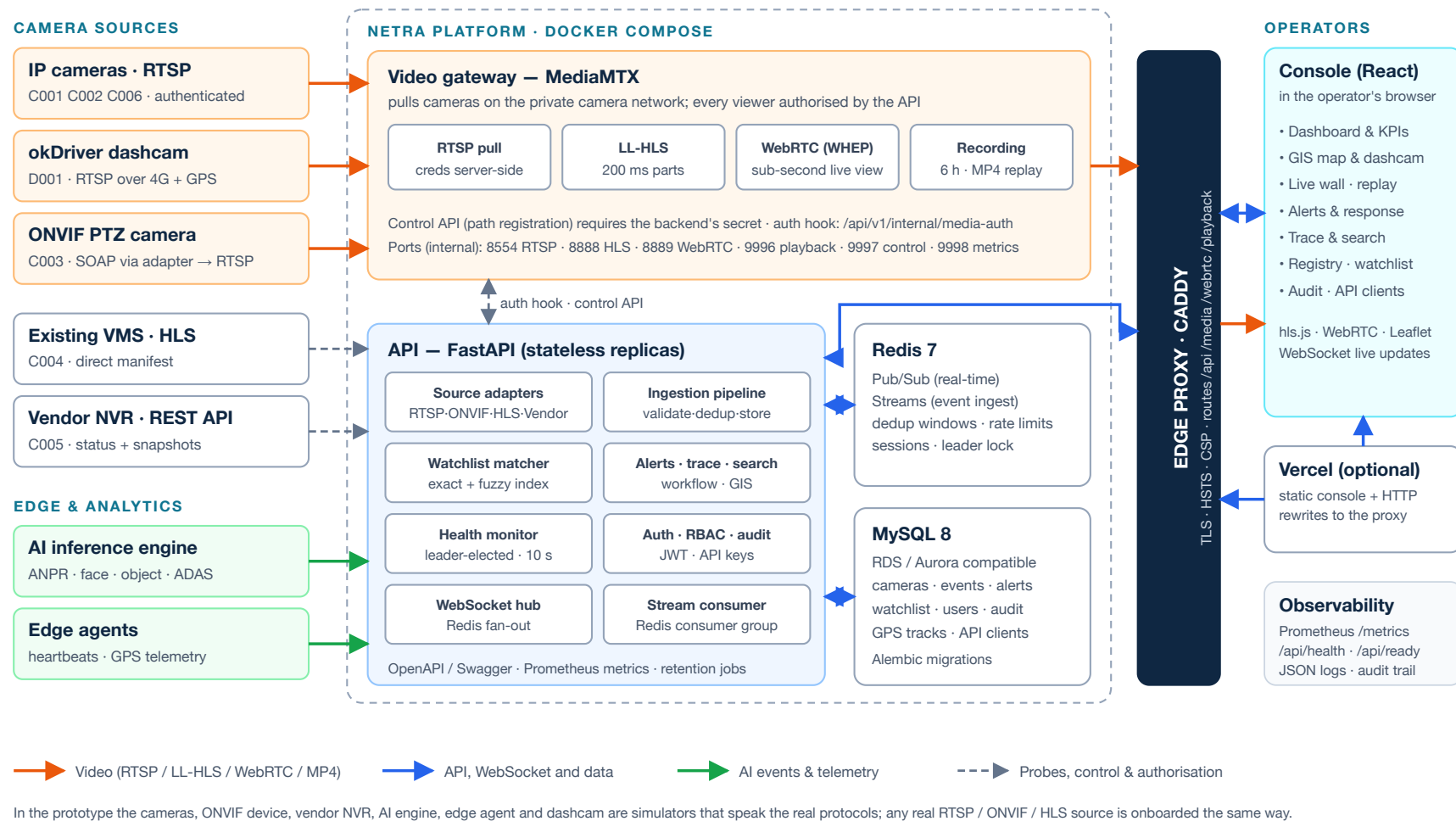


Figure 1 — System architecture

Caddy is the only component exposed to users. The API port and all data stores are internal; the video gateway's control API and viewer access are both authorised by the backend, so no other container or browser can reconfigure or read streams without permission.

4 Components & responsibilities

Component	Responsibility	Key design decisions
Source adapters	One interface (prepare , probe , playback , teardown) for RTSP, ONVIF, HLS and vendor REST sources; register streams with the gateway; resolve ONVIF stream URIs.	New vendor = one class. Credentials decrypted only at the moment of use.
Video gateway	Pull camera RTSP on the camera network; serve LL-HLS and WebRTC; record fMP4; serve MP4 clips by time range.	Viewer auth delegated to the API; on-demand pull available for bandwidth saving.
Ingestion pipeline	Validate, de-duplicate, persist and correlate analytics events; raise or update alerts; publish after commit.	Same code path for REST, batch and Redis Stream input.
Watchlist matcher	Exact and edit-distance-1 matching of plates and person references.	In-memory index; SymSpell deletion index gives O(length) fuzzy lookups; versioned in Redis.
Health monitor	Probe every enabled camera every 10 s or evaluate edge heartbeats; persist, audit and push status changes; run data-retention jobs.	Leader lock in Redis; two-failure hysteresis; heartbeat + video combined.
Real-time hub	Deliver detection.created , alert.* , camera.* , watchlist.changed to every console.	Redis Pub/Sub fan-out: no sticky sessions required.
Stream consumer	Consume the netra:ingest:detections Redis Stream.	Consumer group, ACK after commit, dead-letter stream, reclaim of stale messages.
Auth & audit	JWT sessions, RBAC, API keys, logout, revocation; append-only audit log.	See section 12.
Edge proxy (Caddy)	TLS termination, security headers, routing to API and gateway, blocking internal paths, request-size limits.	Automatic Let's Encrypt in production.
Console (React)	Dashboard, map, live wall, registry, alerts, detections, trace, watchlist, audit, API clients.	TanStack Query cache kept fresh by WebSocket events; error boundary; bundled fonts.
MySQL 8	System of record for cameras, events, alerts, watchlist, users, audit, GPS tracks.	UTC DATETIME(6); composite indexes per query path; Alembic migrations.
Redis 7	Pub/Sub, Streams, dedup windows, rate limits, session revocation, logout counters, leader lock.	Memory-capped; only keys with a TTL are evictable.
Simulators	RTSP camera scenes (FFmpeg), ONVIF device, vendor NVR API, AI analytics engine, edge agent, patrol-van dashcam, scripted scenarios.	Speak the real protocols, so the platform is exercised end to end.

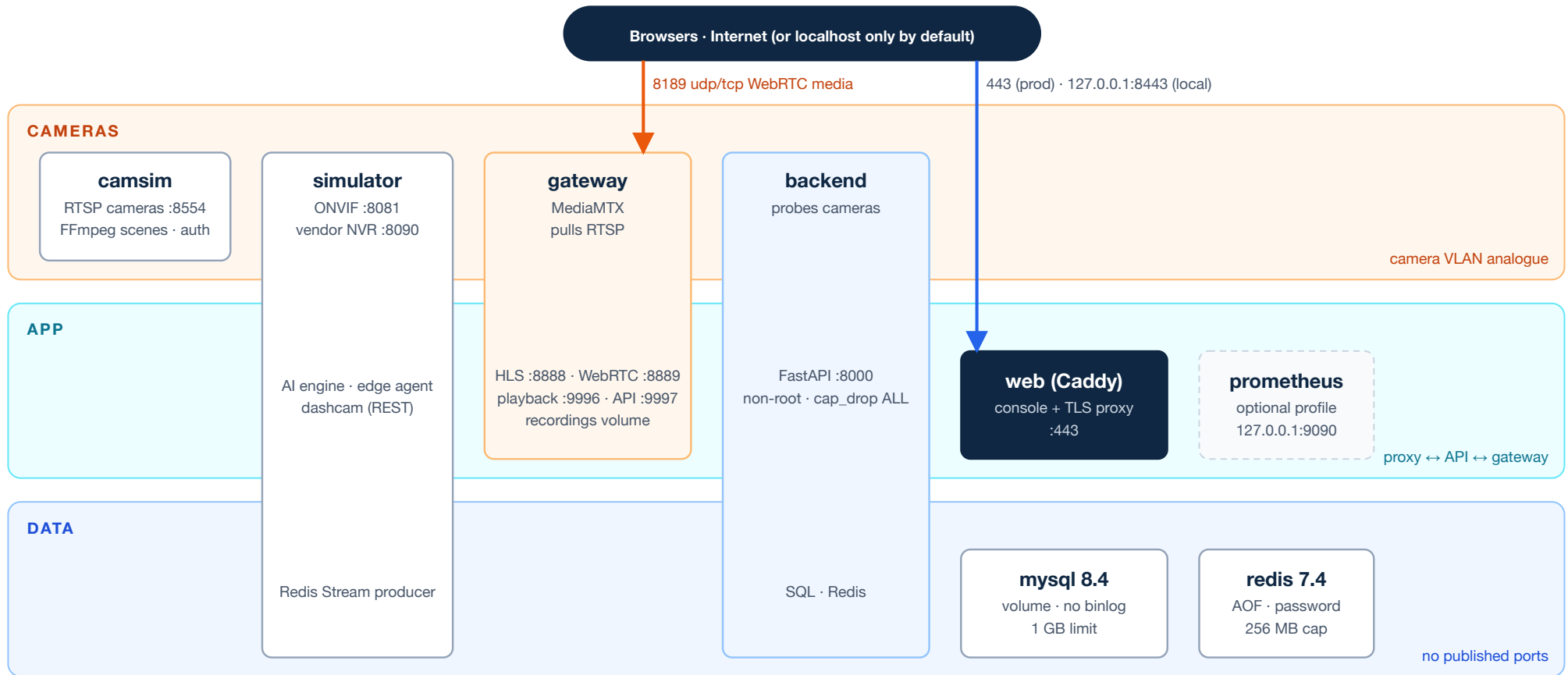
Repository structure

Path	Contents
backend/app/adapters/	RTSP, ONVIF, HLS, vendor-API source adapters
backend/app/api/routes/	auth, cameras, events (search, trace), alerts, watchlist, system (stats, audit, WebSocket)
backend/app/services/	ingestion, matcher, dedup, event bus, stream consumer, health monitor, gateway client, sessions, rate limits, audit

backend/app/core/	configuration, security (tokens, hashing, encryption), SSRF guard, logging
frontend/src/	React console: pages, components (maps, players, alert view), real-time and auth hooks
simulator/sim/	ONVIF mock, vendor NVR, analytics engine, edge agent, dashcam, scenarios
infra/	camera simulator, gateway, Caddy and Prometheus configuration
scripts/ · docs/	environment generation, deployment, backups, outage simulation · architecture, API, security, scalability, deployment

5 Deployment & network topology

Eight containers on three isolated networks that mirror a real deployment: the camera VLAN, the application tier and the data tier. Only the HTTPS proxy and the WebRTC media port are published.



A container spanning several bands is attached to each of those networks.

All containers: restart unless-stopped · no-new-privileges · memory limit · rotated logs (3 × 10 MB). Backend port 8000 is bound to 127.0.0.1 for local debugging only and removed in production.

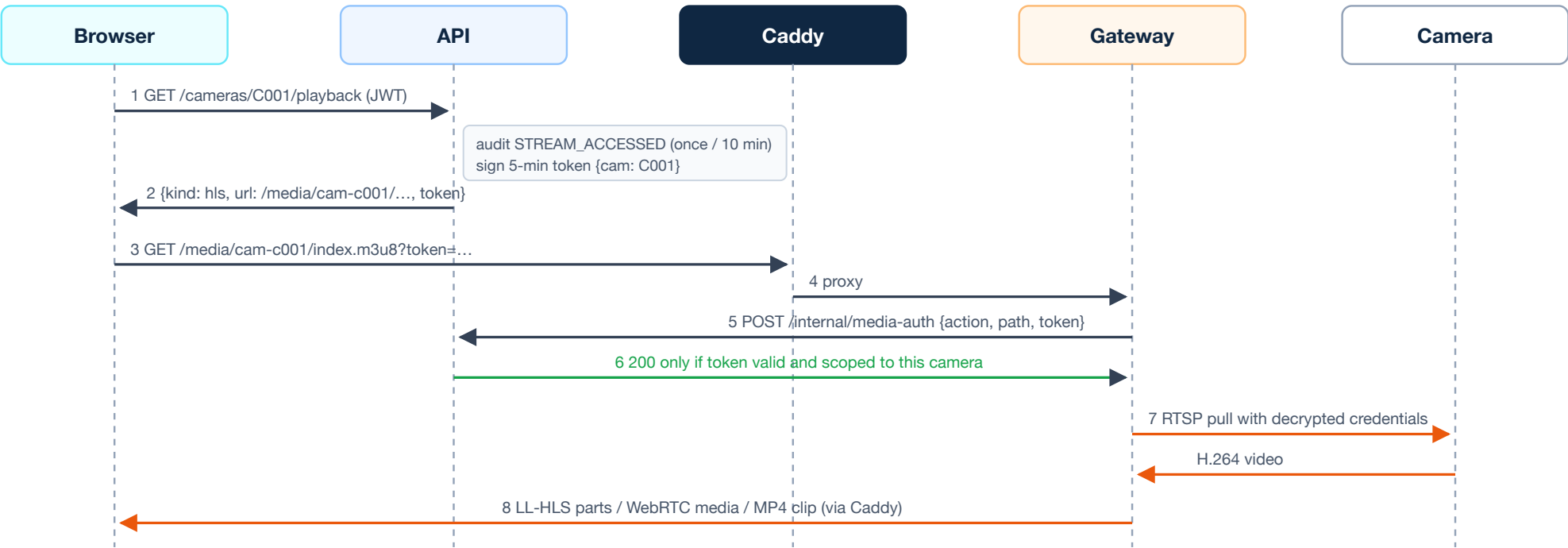
Figure 2 — Containers, networks and published ports

Environments

Environment	How it runs
Local	<code>make up</code> — generates random secrets, builds and starts the stack; console at <code>https://localhost:8443</code> (Caddy internal CA). Ports bound to 127.0.0.1.
Production server	<code>scripts/deploy-vm.sh</code> on any Ubuntu server (x86 or ARM, e.g. an Always-Free cloud VM): Docker, swap, host firewall, automatic security updates, <code>docker-compose.prod.yml</code> overrides (Let's Encrypt on 80/443, one-year HSTS, public WebRTC address), nightly backups.
Vercel (optional)	Static console with HTTP rewrites of <code>/api</code> , <code>/media</code> , <code>/playback</code> , <code>/webrtc</code> to the server; WebSocket connects directly to the server (Vercel cannot proxy WebSockets).
CI	GitHub Actions on every push: backend lint + 38 tests, simulator lint, frontend lint + build, Docker image builds.

6 Camera integration & video path

Five source types, one path to the browser: the gateway relays and records, the API authorises, and no camera credential ever leaves the server.



The browser only ever sees a gateway path and a short-lived, camera-scoped token. Camera credentials are stored Fernet-encrypted, never returned by the API, and inline user:pass@ URLs are rejected. The same check guards WebRTC (WHEP) and recorded playback.

Figure 3 — Stream authorisation sequence

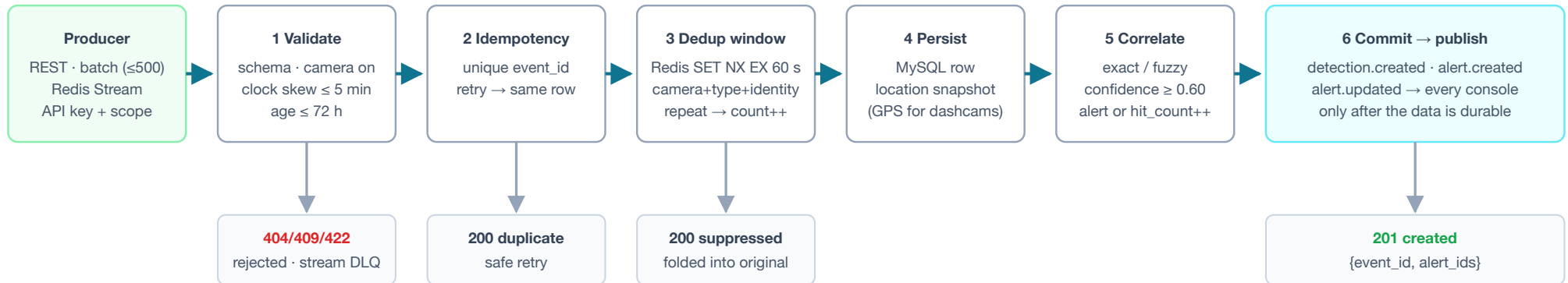
Source	Integration	Health check	Browser playback
RTSP	Gateway pulls <code>rtsp://</code> with credentials injected server-side	Gateway path ready + inbound bitrate $\geq$ 40 kbps	LL-HLS or WebRTC; recorded MP4
ONVIF	SOAP <code>GetProfiles</code> $\rightarrow$ <code>GetStreamUri</code> (WS-Security digest), then RTSP pull	<code>GetDeviceInformation</code> + gateway path	As RTSP
HLS	Existing publisher, no relay needed	Manifest fetch; $>$ slow threshold = DEGRADED	Direct HLS

Vendor API	Proprietary REST: <code>/status</code> , <code>/snapshot.jpg</code> with API key	Vendor status (online / signal / fps)	1 fps snapshot proxy (low bandwidth)
Dashcam	RTSP over 4G + GPS heartbeats every 3 s	Heartbeat age (30 s degraded / 90 s offline) combined with video availability	LL-HLS / WebRTC; live GPS track

Bandwidth: recorded and live video stay inside the platform network; the gateway can pull on demand (only while viewed), and vendor cameras on weak links are served as 1 fps snapshots. At scale, analytics run at the edge and only event metadata is uplinked (section 14).

7 AI event ingestion pipeline

One pipeline serves the REST API, the batch API and the Redis Stream consumer. Each stage has a single job and a clear outcome for the producer.



Measured: 10 repeat reads of one passing vehicle → 1 stored event with repeat_count = 9; a retried POST returns the original event;
a watchlisted plate appears as an alert on every open console in under a second.

Figure 4 — Ingestion stages and outcomes

Event contract

Field	Required	Meaning / validation
event_id	recommended	Producer-side unique id → idempotent retries
camera_id	yes	Must exist and be enabled
event_type	yes	ANPR · VEHICLE_DETECTION · PERSON_DETECTION · FACE_RECOGNITION · OBJECT_DETECTION · OVERSPEED · HARSH BRAKING · COLLISION_WARNING · DRIVER_DROWSINESS
timestamp	yes	ISO-8601 with timezone; ≤ 5 min in the future, ≤ 72 h old
vehicle_number	ANPR	4–12 alphanumeric after normalisation; Indian / BH format flagged
person_ref	face recognition	Gallery identifier matched against person watchlist entries
confidence	yes	0–1; alerts only at ≥ 0.60

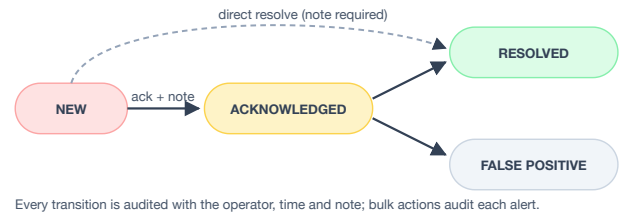
bounding_box	optional	x, y, w, h (pixels or normalised)
latitude / longitude	mobile cameras	Capture position; both or neither; fixed cameras use the registry location
vehicle_type · colour · model_name · snapshot_url · attributes	optional	Enrichment; size-limited

8 Watchlist correlation & alerting

Matching

- **Normalisation:** upper-case, strip spaces and punctuation (`GJ 01 XX 0001` = `GJ01XX0001`).
- **Exact:** dictionary lookup per entity type (vehicle / person).
- **Fuzzy** (vehicles, ≥ 6 characters): SymSpell deletion index finds every entry within edit distance 1 in $O(\text{length})$, independent of watchlist size — covers OCR confusions such as $0 \leftrightarrow O$, $8 \leftrightarrow B$, a dropped character.
- Fuzzy matches are raised one severity lower and labelled *possible match*.
- Expired (`valid_until`) and inactive entries never match; changes propagate to all replicas via a Redis version counter.

Alert life-cycle



Storm control — "prevent duplicate or uncontrolled event generation" (brief §9)

Layer	Mechanism	Effect
Transport retries	Unique <code>event_uid</code>	A retried request or re-delivered stream message never creates a second row
Sensor repeats	Redis <code>SET NX EX</code> per camera + type + identity (60 s)	Many reads of one passing vehicle → one event with a repeat count
Alert storms	Open alert for the same entity at the same camera within 5 min is updated (<code>hit_count</code>)	One actionable alert instead of dozens
Low-quality reads	Minimum confidence 0.60; fuzzy matches downgraded	Fewer false positives reach operators
Abusive producers	Per-API-key rate limit (6,000 / min) and 2 MB body limit	No single source can flood the platform

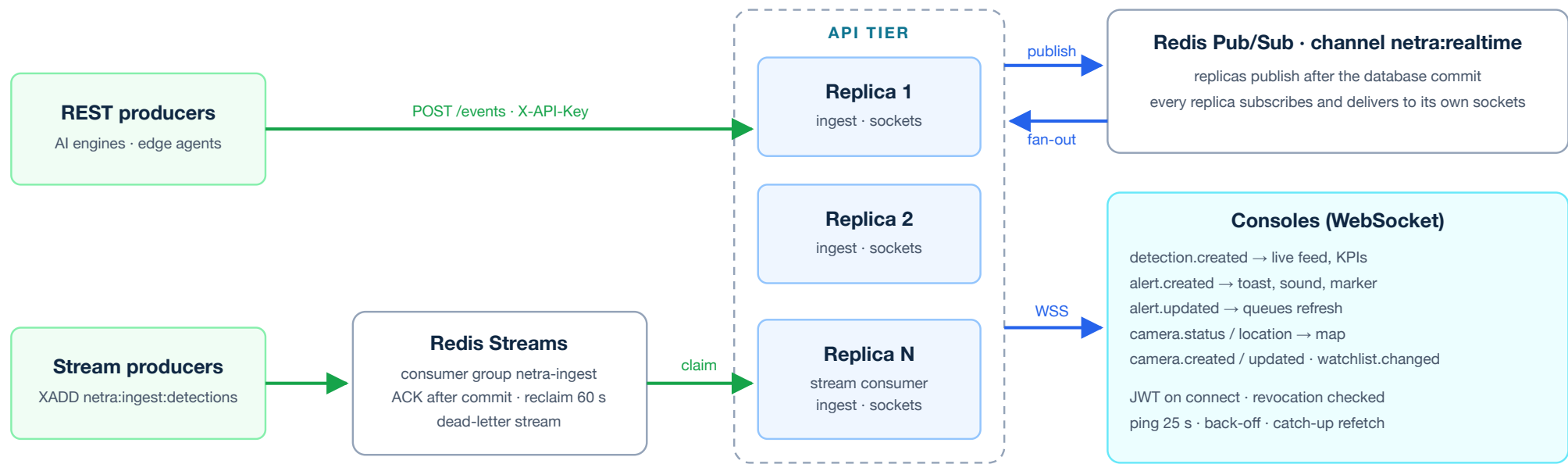
What the operator sees

A toast with sound (triple beep for CRITICAL) and a pulsing map marker appear instantly. The alert view shows the watchlist record (description, make, colour, case reference, agency), the detection in plain language (plate read, confidence, time, camera, address, vehicle seen, number of reads, where the plate was in the frame), warnings for colour mismatch or non-standard plate formats, the camera's live feed, a 40-second recorded evidence clip around the moment of detection, and a response map with the operator's own location, the wanted vehicle's latest sighting, distance and estimated drive time, with a one-click route.

Movement tracing

The trace endpoint returns the most recent 2,000 sightings of an identity in chronological order with leg distance (haversine), elapsed time and implied speed. Two sightings more than 1 km apart at an implied speed above 160 km/h are flagged as **impossible travel — possible cloned or duplicate plate**. The brief's example (GJ01XX0001 at 10:02 → 10:18 → 10:41) is reproduced by `make scenario` and shown as a numbered route on the map.

9 Real-time architecture



Any replica can publish; every replica delivers to its own sockets — no sticky sessions, so the API tier scales behind any load balancer. Without Redis (tests) an in-process bus is used.

Figure 5 — Real-time fan-out and message-channel ingestion

Concern	Design
Consistency	Messages are published only after the database transaction commits, so a console never shows data that does not exist.
Delivery	If Redis publishing fails, the replica still delivers to its local sockets; consoles re-fetch on reconnect.
Back-pressure	Slow sockets are dropped after a 2 s send timeout; the console throttles cache invalidation to one flush per 1.2 s.
Message channel	Redis Streams give at-least-once delivery with consumer groups; at state scale the same contract moves to Kafka partitioned by camera.

10 Data model

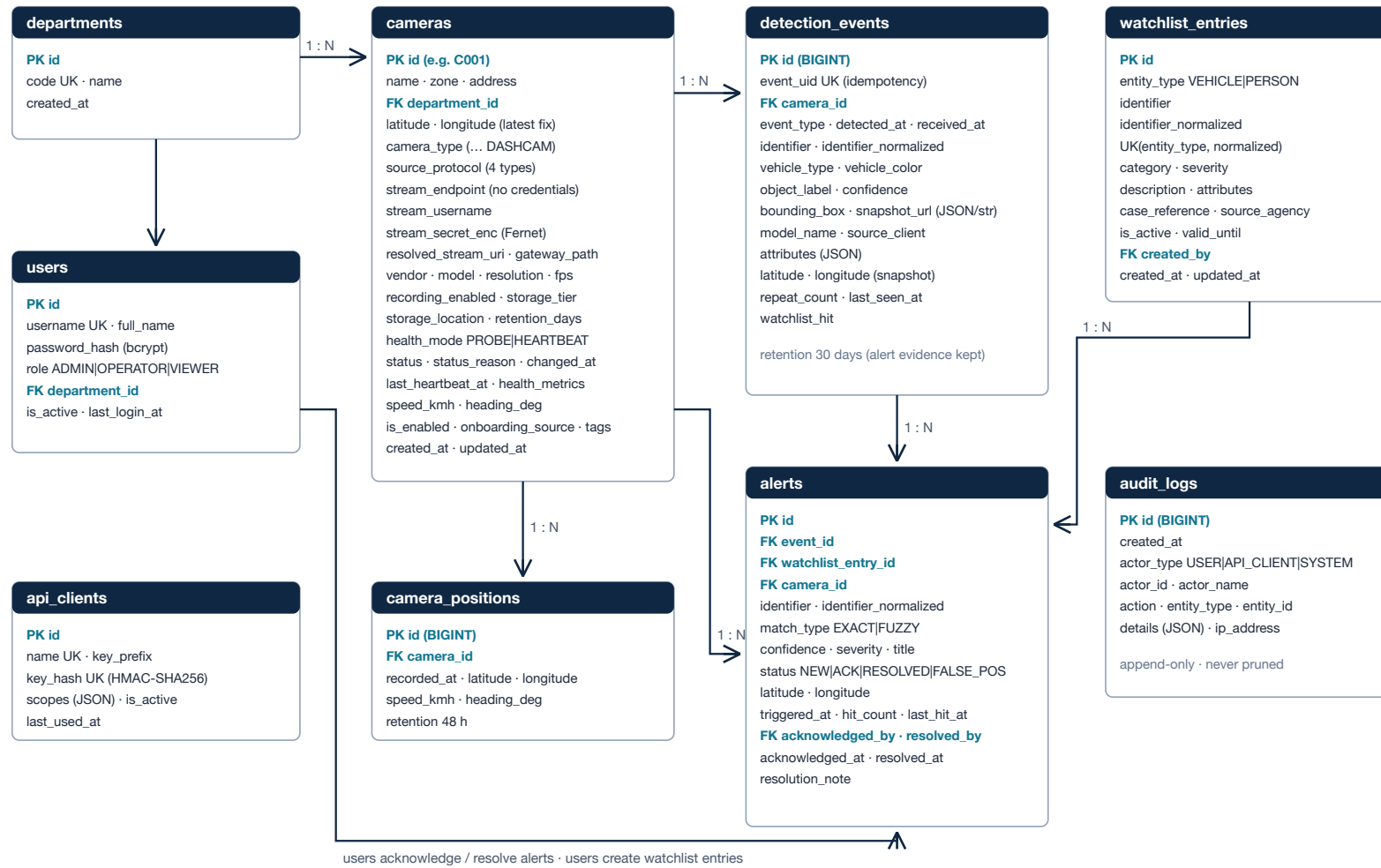


Figure 6 — Entity-relationship diagram (9 tables, MySQL 8, managed by Alembic)

Indexes — one per query path

Table	Index	Serves
detection_events	(identifier_normalized, detected_at)	Entity search and movement trace
detection_events	(camera_id, detected_at)	Per-camera history
detection_events	(event_type, detected_at), (detected_at)	Filtered history, hourly trends, retention
detection_events	UNIQUE (event_uid)	Idempotency
alerts	(status, triggered_at)	Open-alert queue
alerts	(watchlist_entry_id, camera_id, last_hit_at)	Cool-down lookup on the hot path
alerts	(identifier_normalized)	Alerts per entity
cameras	(status), (department_id, zone), (source_protocol), (latitude, longitude)	Registry filters and map
camera_positions	(camera_id, recorded_at)	Dashcam track queries and pruning
watchlist_entries	UNIQUE (entity_type, identifier_normalized), (is_active)	No duplicates; active-list load
audit_logs	(entity_type, entity_id, created_at), (created_at)	Per-object history and audit page

Data conventions

- All timestamps stored as UTC `DATETIME(6)` and returned timezone-aware; enums stored as strings for portable migrations.
- Events keep a **location snapshot** so history stays correct when a camera is moved (and is exact for moving dashcams).
- Watchlist entries are deactivated, never deleted, so past alerts remain explainable; audit logs are never pruned.
- Schema is portable to Amazon RDS / Aurora MySQL without changes; at scale `detection_events` is partitioned by month.

11 API design

RESTful JSON under `/api/v1`, documented with OpenAPI (Swagger UI at `/api/docs`). Consistent pagination (`items`, `total`, `limit`, `offset`), errors as `{"detail": ...}` with field-level validation messages.

Area	Endpoints	Access
Auth & users	POST <code>/auth/login</code> · <code>/auth/token</code> · <code>/auth/logout</code> · GET <code>/auth/me</code> · <code>/users</code> · <code>/departments</code>	public login · any user · admin
Machine identities	GET/POST <code>/api-clients</code> · POST <code>/api-clients/{id}/revoke</code>	admin
Cameras	GET/POST <code>/cameras</code> · POST <code>/cameras/bulk</code> · GET/PATCH <code>/cameras/{id}</code> · GET <code>/cameras/zones</code>	any (read) · admin or API key <code>cameras:write</code>
Camera operations	POST <code>/cameras/{id}/heartbeat</code> · <code>/probe</code> · GET <code>/cameras/{id}/audit</code> · <code>/track</code>	API key <code>cameras:heartbeat</code> · operator · any
Video	GET <code>/cameras/{id}/playback</code> · <code>/snapshot</code> · <code>/recordings</code> · <code>/clip</code> · POST <code>/onvif/discover</code>	any (stream token) · admin
Analytics events	POST <code>/events</code> · <code>/events/batch</code> · GET <code>/events</code> · <code>/events/{id}</code> · <code>/events/export.csv</code>	API key <code>events:write</code> · any · operator
Search & trace	GET <code>/search?q=</code> · <code>/trace/{identifier}</code>	any
Watchlist	GET/POST <code>/watchlist</code> · PATCH/DELETE <code>/watchlist/{id}</code>	any (read) · operator
Alerts	GET <code>/alerts</code> · <code>/alerts/{id}</code> · POST <code>/alerts/{id}/acknowledge</code> · <code>/resolve</code> · <code>/alerts/bulk</code>	any (read) · operator
Stats & audit	GET <code>/stats/overview</code> · <code>/stats/timeseries</code> · <code>/audit</code>	any · admin
Real-time	WS <code>/ws?token=...</code>	any user
System	GET <code>/api/health</code> · <code>/api/ready</code> · <code>/api/metrics</code> (internal)	probes · Prometheus

Example — one backend workflow

#	Request	Result
1	POST <code>/api/v1/events</code> X-API-Key: <code>ntr_...</code> <code>{"event_id":"edge-42-187","camera_id":"C001","event_type":"ANPR","timestamp":"2026-09-26T10:02:11+05:30","vehicle_number":"GJ01XX0001","confidence":0.94}</code>	201 <code>{event_id, alert_ids:[...]}</code> — watchlisted stolen vehicle
2	Every console receives <code>alert.created</code> over the WebSocket	Toast, sound, marker pulse, queue entry
3	POST <code>/api/v1/alerts/{id}/acknowledge</code> <code>{"note":"PCR 12 dispatched"}</code>	ACKNOWLEDGED, audited, pushed to all consoles
4	GET <code>/api/v1/trace/GJ01XX0001</code>	Chronological stops, distances, speeds, anomalies

12 Security architecture

Defence in depth: each layer assumes the one in front of it may fail.

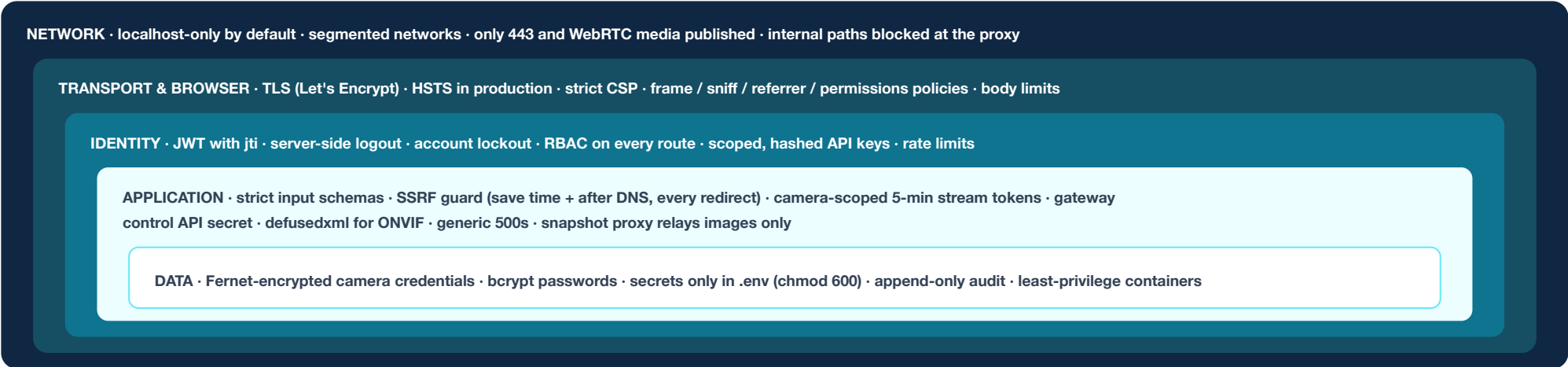


Figure 7 — Security layers

Brief §13 requirement	Implementation
Authentication & RBAC	JWT (HS256, 8 h, required exp/sub/typ, unique jti); roles ADMIN / OPERATOR / VIEWER enforced per route; tokens of one type (stream vs access) rejected elsewhere; logout and password change revoke tokens server-side; 5 failed logins lock an account for 15 minutes.
Input validation & secure API	Pydantic schemas with length limits on every field, per-protocol URL schemes, coordinates, timestamps with timezone, payload size; semantic checks (camera enabled, skew, age); non-nullable fields protected on update.
Stream URLs & credentials	Credentials encrypted at rest and never serialised; inline credentials rejected; gateway injects them server-side; browsers receive camera-scoped 5-minute tokens verified by the gateway for live, WebRTC and recorded access.
HTTPS / TLS	Caddy terminates TLS with automatic Let's Encrypt certificates in production; one-year HSTS in production.
Audit logs	Logins (success, failure, logout), logout, onboarding, field-level edits with secrets masked, status changes, stream and recording access, alert handling, watchlist and API-key changes, password resets.
Rate limiting	Redis fixed windows: login per IP, API per user (600 / min), ingest per API key (6,000 / min).
No secrets in Git	<code>.env</code> is git-ignored and generated with random values; staged files are scanned against real secret values before commit.
Environment configuration	All settings via environment variables (pydantic-settings); the backend refuses to start in production with a default JWT secret.

Supply chain

Pinned dependencies; `pip-audit` and `npm audit` report 0 known vulnerabilities; CI builds every image.

Verification: 38 automated tests include logout, server-side logout, forged and wrong-type tokens, seven SSRF payloads (Redis, MySQL, localhost, IPv6 loopback, cloud metadata), scope validation and security headers; live checks confirmed that requests without, or with another camera's, stream token are refused and that internal endpoints are unreachable through the proxy.

13 Reliability & operations

Failure mode	Protection
Process crash	<code>restart: unless-stopped</code> on every container; Docker enabled at boot; health checks on API, MySQL and Redis; the API waits for healthy dependencies before starting.
Camera / stream failure	Probes every 10 s; two-failure hysteresis against flapping; heartbeat-mode devices are DEGRADED when alive but not streaming; gateway paths self-heal after a gateway restart; status changes audited and pushed live.
Memory exhaustion	Per-container memory limits; Redis <code>maxmemory</code> evicting only TTL keys (never the ingest stream); swap on servers.
Disk exhaustion	Docker log rotation (3 × 10 MB); MySQL binary logs off; events pruned after 30 days (alert evidence kept); GPS tracks after 48 h; recordings after 6 h; stream lengths capped; weekly image prune.
Data loss	Nightly consistent <code>mysqldump</code> (7 days kept); named volumes; Redis AOF.
UI errors	React error boundary with recovery; WebSocket reconnect with exponential back-off and catch-up refetch.
Certificates & patches	Automatic certificate renewal; unattended security updates on the server.

Observability

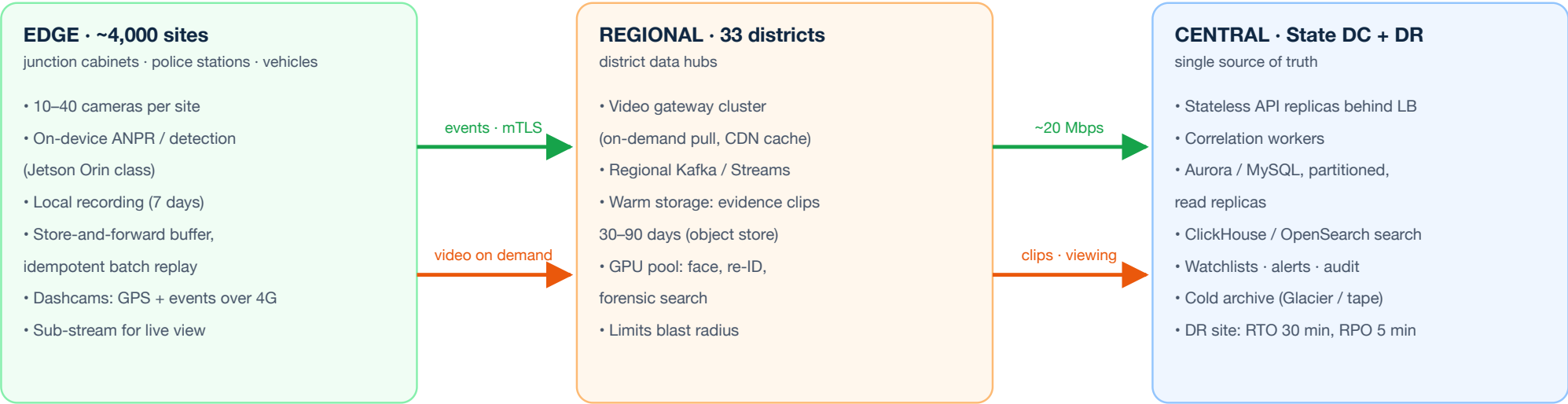
Signal	Detail
Health	<code>/api/health</code> (liveness), <code>/api/ready</code> (database + Redis), dashboard service strip (API, DB, Redis, gateway, ingest backlog, ready streams, viewers).
Metrics	Prometheus: <code>netra_events_total{type,result}</code> , <code>netra_alerts_total</code> , <code>netra_ingest_seconds</code> histogram, <code>netra_cameras{status}</code> , <code>netra_ws_connections</code> ; gateway metrics on :9998.
Logs & audit	Structured JSON logs with request IDs; business audit trail in the database.
SLO targets	Detection-to-alert p95 < 2 s · camera status freshness < 30 s · ingest error rate < 0.1 %.

Quality

38 backend tests (auth & RBAC, onboarding, validation, idempotency, dedup, exact/fuzzy correlation, cool-down, alert workflow and bulk actions, trace, stats, dashcam telemetry, security, retention); Ruff lint and format; TypeScript strict mode and ESLint; GitHub Actions on every push; Docker image builds in CI.

14 Scalability to 80,000 cameras

80,000 cameras × 2 Mbps ≈ 160 Gbps if everything were sent centrally. The design instead keeps video near the camera and moves metadata: an ANPR event is ~0.6 KB, so 4,000 events/s is only ~20 Mbps into the central tier.



The prototype already has these seams: adapters (edge / vendor integration), batch API (store-and-forward), Redis Streams consumer groups (→ Kafka), heartbeat-mode health, on-demand gateway pulls.

Figure 8 — Central, regional and edge tiers

Sizing (assumptions: 30 % ANPR cameras, 10 vehicles/min/camera after edge dedup, peak ×3)

Metric	Value
Average / peak events	≈ 4,000 / 12,000 per second
Rows per day	≈ 345 million (≈ 200 GB raw)
Ingest replicas at peak	20–40 (batched inserts)
Edge accelerators	~1,800 Orin NX (12–16 streams each) or 600–800 T4-equivalents centrally
Central viewing budget	2,000 sub-streams × 0.5 Mbps ≈ 1 Gbps

Scaling levers

Concern	Approach
API	Stateless replicas + HPA on CPU and queue lag
Ingest	Kafka partitioned by camera; bulk-insert workers
Database	Monthly partitions, read replicas, regional shards
Search	ClickHouse / OpenSearch fed by the event stream
Video	H.265, sub-streams, on-demand pull, snapshots on 2G/4G
Storage	Hot (edge, 7–15 d) · warm (regional, 30–90 d) · cold (1–7 y)
HA / DR	Multi-AZ DB, RF-3 Kafka, DR site; RTO 30 min, RPO 5 min

Continuous recording	≈ 860 TB/day → kept at the edge; only evidence promoted
----------------------	---

Security	mTLS edge, SSO + MFA, Vault, WORM audit, DPDP Act
----------	---

Indicative cost (state-wide, hybrid model, 2026 list prices)

Item	Estimate
Edge AI boxes (1,800) · district hub servers (33 × 3)	≈ ₹15 Cr + ₹25 Cr one-time
Central compute, warm and cold storage, network, 25–30 FTE operations	≈ ₹18–22 Cr per year
Total	≈ ₹40 Cr one-time + ₹18–22 Cr/year ≈ ₹2,500 per camera per year

15 Technology, cost & roadmap

Layer	Technology	Why
Backend	Python 3.12, FastAPI, SQLAlchemy 2 (async), Pydantic v2, Alembic	okDriver's stack; async I/O for probes and WebSockets; automatic OpenAPI
Database	MySQL 8.4	RDS / Aurora compatible, as preferred in the brief
Real-time	Redis 7 (Pub/Sub, Streams), WebSocket, WebRTC	Horizontal fan-out; message channel with consumer groups
Video	MediaMTX, FFmpeg, hls.js	Production-grade RTSP relay, LL-HLS, WebRTC, recording
Frontend	React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, Leaflet, Recharts	Operational console, strict typing, live cache
Edge & deploy	Caddy, Docker Compose, GitHub Actions, optional Vercel	Automatic HTTPS, one-command setup, CI on every push
Maps	OpenStreetMap tiles	Free with attribution; self-hostable for air-gapped use

COST ₹0 Every component is free and open source (MIT / BSD / Apache-2.0 / GPL community editions); certificates from Let's Encrypt are free; no paid API key or subscription is required to build, run or deploy the prototype.

Further detail in the repository: [README.md](#) , [docs/ARCHITECTURE.md](#) , [docs/API.md](#) , [docs/SECURITY.md](#) , [docs/SCALABILITY.md](#) , [docs/DEPLOYMENT.md](#) , [docs/DEMO_SCRIPT.md](#) .

Known limitations of the prototype

- AI inference and camera feeds are simulated (the brief does not require model training); real models post to the same API.
- ONVIF discovery probes configured hosts rather than WS-Discovery multicast, which does not cross container networks.
- Recording is a 6-hour hot tier; warm and cold tiers are designed (section 14) but not implemented.
- Department-scoped data access is modelled (departments on users and cameras) but not yet enforced per query.

Roadmap

#	Next step	Value
1	Real edge inference container (YOLO + OCR on DeepStream)	Live ANPR from actual footage into the same API / stream
2	Evidence clips promoted to object storage with legal hold	Chain of custody beyond the 6-hour hot tier
3	SSO (OIDC) + MFA and department-scoped access	Multi-agency tenancy and stronger identity
4	Kafka + ClickHouse event backbone; partitioned MySQL	State-scale throughput and months of searchable history
5	PTZ control over ONVIF; cross-camera re-identification	Operator control and tracking beyond plates
6	End-to-end browser tests (Playwright)	Regression safety for operator workflows